

See also

[Console](#)

[clearConsole \[SimTalk\]](#)

Video on YouTube

<https://youtu.be/MXDcx2yplxc?si=7AMwbDavKOHYRNRA&t=245>

Functions for Converting Data Types

Functions for Converting Data Types

SimTalk provides these functions for converting data types:

- [Converting Numerical Values](#)
- [Converting Physical Data Types](#)
- [Converting Physical Data Types with Units into Data Types without Units](#)
- [Converting Values without Units into Physical Data Types with Units](#)
- [Converting References](#)
- [Converting Time and Date Values](#)

Note

You can use the function [to_str \[SimTalk\]](#) to convert any value to a string.

Converting Numerical Values

Converting Numerical Values

SimTalk provides the functions listed in the table of contents to the left for converting numerical values.

bool_to_num [SimTalk]

Converts the specified boolean value into a number of data type *integer*.

Type

Function

Syntax

```
bool_to_num(Value:boolean) → integer
```

Parameter

The parameter *Value* has the data type *boolean*.

Return Value

The return value has the data type *integer*.

true returns 1.

false returns 0.

Example

```
var gross: real
var inclVAT: boolean
print bool_to_num(true)
print "Price:"
print gross + 0.16*gross*bool_to_num(inclVAT)
```

bool_to_str [SimTalk]

Converts the specified boolean value into a *string*.

Type

Function

Syntax

```
bool_to_str(Value:boolean) → string
```

Parameter

The parameter *Value* has the data type *boolean*.

Return Value

The return value has the data type *string*.

Example

```
var s: string  
var b: boolean  
s := bool_to_str(b)
```

num_to_bool [SimTalk]

Converts the specified number into a *boolean* value.

Type

Function

Syntax

```
num_to_bool(Value:real) → boolean
```

Parameter

The parameter *Value* has the data type *real*.

Return Value

The return value has the data type *boolean*.

true if the parameter is not zero.

false if the parameter is zero.

Example

```
var s: stack[boolean]  
var number: real  
stack.create  
stack.insert(num_to_bool(number))
```

num_to_hex [SimTalk]

Converts the specified integer number into a *string* value.

Remarks

num_to_hex only shows the integer value as a hexadecimal value.

Type

Function

Syntax

```
num_to_hex(Value:integer[, Is64Bit:boolean:=false]) → string
```

Parameter

You can specify the following parameters:

- The parameter *Value* of data type *integer* designates the value that you want to convert.
- The optional parameter *Is64Bit* of data type *boolean* set if *Plant Simulation* is to create a 64-Bit hexadecimal number (*true*) or a 32-Bit hexadecimal number (*false*).

To ensure that the function *num_to_hex* is compatible with previous versions of *Plant Simulation*, it creates a 32-Bit hexadecimal number if you do not specify the parameter.

Default Value of the Parameter

The default values is *false*.

Return Value

The return value has the data type *string*.

Example

```

print num_to_hex(-1)           // "ffffffff"
print num_to_hex(-1, true)     // "ffffffffffffffffffff"
print num_to_hex(2343432205, true) // "8badf00d"

```

num_to_str [SimTalk]

Converts the specified real number into a *string*.

Type

Function

Syntax

```
num_to_str(Number:real[, Precision:integer, Width:integer]) -> string
```

Parameter

You can specify the following parameters:

- The parameter *Number* of data type *real* designates the number that is going to be converted.
- For the numerical data types the optional parameter *Precision* of data type *integer* designates the precision, i.e., the number of significant digits, which the resulting string has at most.

If the data type of the parameter *Number* is *integer* or *time*, the parameter *Precision* has no function.

You can also specify a negative value for the parameter *Precision*. The absolute number of the negative value will then be used as the number of desired decimal places.

- The optional parameter *Width* of data type *integer* designates the desired number of characters. If this number is not reached, *Plant Simulation* adds zeros (0) at the start of the string.

Return Value

The return value has the data type *string*.

Example

```
print num_to_str(3, 0, 4)    // returns 0003
print num_to_str(pi, -2, 6)  // returns 003.14
```

str_to_bool [SimTalk]

Converts the specified string into a value of data type *boolean*.

Remarks

The spelling of the string is not case-sensitive.

Type

Function

Syntax

```
str_to_bool(Value:string) → boolean
```

Parameter

The parameter *Value* has the data type *string*.

Return Value

The return value has the data type *boolean*.

true if the value true was passed.

false for any other value.

Example

```
table[1,1] := str_to_bool("true")
```

str_to_num [SimTalk]

Converts the specified string value into a floating-point value of data type *real*.

Remarks

Apart from digits the string can optionally contain leading white spaces, a sign, a decimal point and an e (lower case e) or an E (upper case E) for the exponent. If the string begins with 0x, the following string is interpreted as a hexadecimal integer number.

If the string contains an invalid character, the function *str_to_num* only converts all preceding characters into a number.

Type

Function

Syntax

```
str_to_num(Value:string) → real
```

Parameter

The parameter *Value* of data type *string* designates the data that you want to convert.

Return Value

The return value has the data type *real*.

Examples

```
x := str_to_num("  -7")           // minus seven
```

```
y := str_to_num("3.141xyz")       // 3.141
```

```
w := str_to_num("-1e6")           // minus one million
```

```
var n: integer := str_to_num("0x10a") // hexadecimal number 10a  
print n                               // outputs 266
```

time_to_str [SimTalk]

Converts the specified time value into a *string*.

Type

Function

Syntax

```
time_to_str(Value:time[, FormatLikeDialogs:boolean]) → string
```

Parameters

You can specify the following parameters:

- The parameter *Time* of data type *time* designates the time that is going to be converted.
- The optional parameter *FormatLikeDialogs* of data type *boolean* sets if *Plant Simulation* uses the same time format as in text boxes of the dialogs (*true*). If you do not enter a value for the parameter or if you set it to *false*, *Plant Simulation* uses the same time format as in *tables*, which shows four decimal places.

Return Value

The return value has the data type *string*.

Example

```
var t: time  
t := 60  
Dialog.setCaption("myProcTime", time_to_str(t, true))
```

to_str [SimTalk]

Converts all of the specified parameters into a *string* each, concatenates the resulting *string* into a single *string*, and then returns this *string*.

Type

Function

Syntax

```
to_str(Parameter1:any[, Parameter2:any, ...]) → string
```

Parameters

The parameters named *Parameter* have the data type *any*.

Return Value

The return value has the data type *string*.

Examples

```
var s: string
var x: real
s := to_str(true)           // assigns "true"
x := 1.5
s := to_str("length: ", x, " meters") // assigns "length: 1.5 meters"
```

```
var a : real[]
var s : string := to_str(a) // s := "[ ]"
```

```
var a : string["Kent, Clark", ""]
var s : string := to_str(a) -- assigns ["Kent, Clark", ""]
```

See also

[datetime_to_str \[SimTalk\]](#)

Converting Physical Data Types

Converting Physical Data Types

SimTalk provides the functions listed in the table of contents to the left for converting strings to physical data types.

You can also enter a unit which the function then uses.

str_to_acceleration [SimTalk]

Converts the specified data into a number of data type *acceleration*.

Remarks

You can specify one of these units into which the function converts the *string*: "mm/s²", "cm/s²", "m/s²", "km/h²", "m/min²", "in/s²", "ft/s²", or "yd/s²".

If you do not specify a unit, *Plant Simulation* outputs the acceleration according to the settings you selected under **File > Model Settings/Preferences > Units > Acceleration**.

Suppose that *Plant Simulation* outputs the acceleration in a dialog box. *Plant Simulation* outputs the acceleration according to the settings you selected under **File > Model Settings/Preferences > Units > Acceleration**. When evaluating the value, *Plant Simulation* has to reconvert it to m/s². The function *str_to_acceleration* reconverts and re-translates the value in a single step.

Type

Function

Syntax

```
str_to_acceleration(Data:string) → acceleration
```

Parameter

The parameter *Data* of data type *string* designates the data that you want to convert.

Return Value

The return value has the data type *acceleration*.

Examples

```
// reads values in dialog box  
var entry: string
```

```
entry := dialog.getValue("acceleration")  
print str_to_acceleration(entry)
```

```
str_to_acceleration("20")  
// interprets 20 according to the acceleration setting you selected under  
Preferences
```

```
str_to_length("20m/ss")  
// interprets 20 as meters per square second
```

See also

[Acceleration \[preferences\]](#), Unit

str_to_length [SimTalk]

Converts the specified data into a number of data type *length*.

Remarks

You can specify one of these units into which the function converts the *string*: "mm", "m", "cm", "km", "in", "ft", "yd", or "mi".

If you do not specify a unit, *Plant Simulation* outputs the acceleration values according to the settings you selected under **File > Model Settings/Preferences > Units > Length**.

Suppose *Plant Simulation* outputs the length in a dialog box. *Plant Simulation* outputs the length according to the settings you selected under **File > Model Settings/Preferences > Units > Length**. When evaluating the value, *Plant Simulation* has to reconvert it to m (meters) again. The function by *str_to_length* reconverts and re-translates the value in a single step.

Type

Function

Syntax

```
str_to_length(Data:string) → length
```

Parameter

The parameter *Data* of data type *string* designates the data that you want to convert.

Return Value

The return value has the data type *length*.

Examples

```
// reads the values in the dialog box
var entry: string
entry := dialog.getValue("length")
print str_to_length(entry)
```

```
entry := dialog.getValue("width")  
print str_to_length(entry)
```

```
str_to_length("1.23")  
// interprets 1.23 according to the length setting you selected under  
Preferences
```

```
str_to_length("1.23ft")  
// interprets 1.23 as feet
```

See also

[Length \[preferences\]](#), Unit

str_to_speed [SimTalk]

Converts the specified data into a number of data type *speed*.

Remarks

You can specify one of these units into which the function converts the *string*: "mm/s", "cm/s", "m/s", "km/h", "m/min", "in/s", "ft/s", "yd/s", or "mph".

If you do not specify a unit, *Plant Simulation* outputs the acceleration according to the settings you selected under **File > Model Settings/Preferences > Units > Speed**.

Suppose *Plant Simulation* outputs the speed in a dialog box. *Plant Simulation* outputs the speed according to the settings you selected under **File > Model Settings/Preferences > Units > Speed**. When evaluating the value, *Plant Simulation* has to reconvert it to **m/s** (meter per second). The function *str_to_speed* reconverts and re-translates the value in a single step.

Type

Function

Syntax

```
str_to_speed(Data:string) → speed
```

Parameter

The parameter *Data* of data type *string* designates the data that you want to convert.

Return Value

The return value has the data type *speed*.

Examples

```
// reads values in dialog box  
var entry : string
```

```
entry := dialog.getValue("move forward")  
print str_to_speed(entry)
```

```
str_to_speed("10")  
// interprets 10 according to the speed setting you selected under  
Preferences
```

```
str_to_speed("10m/ss")  
// interprets 10 as meters per square second
```

See also

[Speed \[preferences\]](#), Unit

str_to_weight [SimTalk]

Converts the specified data into a number of data type *weight*.

Remarks

You can specify one of these units into which the function converts the *string*: "g", "kg", "t", "lb", or "oz".

If you do not specify a unit, *Plant Simulation* outputs the acceleration according to the settings you selected under **File > Model Settings/Preferences > Units > Mass**.

Suppose *Plant Simulation* outputs the weight in a dialog box. *Plant Simulation* outputs the weight according to the settings you selected under **File > Model Settings/Preferences > Units > Mass**. When evaluating the value, *Plant Simulation* has to reconvert it to kg (kilograms). The function *str_to_weight* reconverts and re-translates the value in a single step.

Type

Function

Syntax

```
str_to_weight(Data:string) → weight
```

Parameter

The parameter *Data* of data type *string* designates the data that you want to convert.

Return Value

The return value has the data type *weight*.

Examples

```
// reads values in dialog box  
var entry: string  
entry := dialog.getValue("net")  
print str_to_weight(entry)
```

```
entry := dialog.getValue("gross")
print str_to_weight(entry)
```

```
str_to_weight("10")
// interprets 10 according to the mass setting you selected under
Preferences
```

```
str_to_weight("10kg")
// interprets 10 as kilograms
```

See also

[Mass \[preferences\]](#), [Unit](#)

Converting Data Types with Units into Data Types without Units

Converting Physical Data Types with Units into Data Types without Units

SimTalk provides the functions listed in the table of contents to the left for converting physical data types with units into data types without units.

This is especially handy if you want to assign incompatible physical data types, which would normally output a warning to the *Console*.

Example

```
var w: weight
// assign a time to a weight without showing a warning
w := time_to_num(EventController.simTime)
```

See also

[Converting Values without Units into Physical Data Types with Units](#)

acceleration_to_num [SimTalk]

Converts the specified acceleration into a value without a unit.

Remarks

acceleration_to_num is handy if you want to assign a local variable to another local variable which has a different unit. Normally this is forbidden. Using *acceleration_to_num* you can discard the unit.

Type

Function

Syntax

```
acceleration_to_num(Value:acceleration[, Unit:string="m/s²"]) → real
```

Parameters

You can specify the following parameters:

- The parameter *Data* has the data type *acceleration*.
- The optional parameter *Unit* of data type *string* designates the physical value which is going to be converted to a number without a unit by using the specified unit.

You can specify one of these units: "mm/s²", "cm/s²", "m/s²", "km/h²", "m/min²", "in/s²", "ft/s²", or "yd/s²".

Default Value of the Parameter

The default value is "m/s²".

Return Value

The return value has the data type *real*.

Example

```
var t: time, a: acceleration  
t := acceleration_to_num(a)
```

See also

[num_to_acceleration \[SimTalk\]](#)

length_to_num [SimTalk]

Converts the specified length into a value without a unit.

Type

Function

Syntax

```
length_to_num(Data:length[, Unit:string="m"]) → real
```

Parameters

You can specify the following parameters:

- The parameter *Data* of data type *length* designates the value that you want to convert.
- The optional parameter *Unit* of data type *string* designates the physical value which is going to be converted to a number without a unit by using the specified unit.

You can specify one of these units: "mm", "cm", "m", "km", "in", "ft", "yd", or "mi".

Default Value of the Parameter

The default value is "m".

Return Value

The return value has the data type *real*.

Examples

```
var t: time, le: length
t := length_to_num(le)
```

```
Conveyor.Length = 2yd
print length_to_num(Conveyor.Length, "ft") // 6
```

See also

[num_to_length \[SimTalk\]](#)

speed_to_num [SimTalk]

Converts the specified speed into a value without a unit.

Type

Function

Syntax

```
speed_to_num(Data:speed[, Unit:string="m/s"]) → real
```

Parameters

You can specify the following parameters:

- The parameter *Data* has the data type *speed*.
- The optional parameter *Unit* of data type *string* designates the physical value which is going to be converted to a number without a unit by using the specified unit.

You can specify one of these units: "mm/s", "cm/s", "m/s", "km/h", "m/min", "in/s", "ft/s", "yd/²", or "mph".

Default Value of the Parameter

The default value is "m/s".

Return Value

The return value has the data type *real*.

Example

```
var t: time, s: speed  
t := speed_to_num(s)
```

See also

[num_to_speed \[SimTalk\]](#)

time_to_num [SimTalk]

Converts the specified time into a value without a unit.

Type

Function

Syntax

```
time_to_num(Data:time) → real
```

Parameter

The parameter *Data* has the data type *time*.

Return Value

The return value has the data type *real*.

Example

```
var w: weight, t: time  
w := time_to_num(t)
```

weight_to_num [SimTalk]

Converts the specified weight into a value without a unit.

Type

Function

Syntax

```
weight_to_num(Data:weight[, Unit:string="kg"]) → real
```

Parameters

You can specify the following parameters:

- The parameter *Data* has the data type *weight*.
- The optional parameter *Unit* of data type *string* designates the physical value which is going to be converted to a number without a unit by using the specified unit.

You can specify one of these units: "g", "kg", "t", "lb", or "oz".

Default Value of the Parameter

The default value is "kg".

Return Value

The return value has the data type *real*.

Example

```
var t: time, w: weight  
t := weight_to_num(w)
```

See also

[num_to_weight \[SimTalk\]](#)

Converting Values without Units into Data Types with Units

Converting Values without Units into Physical Data Types with Units

SimTalk provides the functions listed in the table of contents to the left for converting a value of data type *real* or *integer* into a value with a physical unit.

See also

[Converting Physical Data Types with Units into Data Types without Units](#)

num_to_acceleration [SimTalk]

Converts a specified number without a unit into a value of data type *acceleration*.

Type

Function

Syntax

```
num_to_acceleration(Number:real[, Unit:string="m/s2"]) → acceleration
```

Parameters

You can specify the following parameters:

- The parameter *Number* of data type *real* designates the number without a unit which is going to be converted.
- The optional parameter *Unit* of data type *string* designates the physical unit which going to be assigned while converting.

You can specify one of these units: "mm/s²", "cm/s²", "m/s²", "km/h²", "m/min²", "in/s²", "ft/s²", or "yd/s²".

Default Value of the Parameter

The default value is "m/s²".

Return Value

The return value has the data type *acceleration*.

Example

```
var s: speed  
s := num_to_acceleration(3.7) * Conveyor3.Time
```

See also

[acceleration_to_num \[SimTalk\]](#)

num_to_length [SimTalk]

Converts the specified number without a unit into a value of data type *length*.

Type

Function

Syntax

```
num_to_length(Number:real[, Unit:string="m"]) → length
```

Parameters

You can specify the following parameters:

- The parameter *Number* of data type *real* designates the number without a unit which is going to be converted.
- The optional parameter *Unit* of data type *string* designates the physical unit which going to be assigned while converting.

You can specify one of these units: "mm", "cm", "m", "km", "in", "ft", "yd", or "mi".

Default Value of the Parameter

The default value is "m".

Return Value

The return value has the data type *length*.

Example

```
// Set conveyor length to 1 ft:  
Conveyor.Length = 1ft  
Conveyor.Length = num_to_length(1, "ft")
```

See also

[length_to_num \[SimTalk\]](#)

num_to_speed [SimTalk]

Converts the specified number without a unit into a value of data type *speed*.

Type

Function

Syntax

```
num_to_speed(Number:real[, Unit:string="m/s"]) → speed
```

Parameters

You can specify the following parameters:

- The parameter *Number* of data type *real* designates the number without a unit which is going to be converted.
- The optional parameter *Unit* of data type *string* designates the physical unit which going to be assigned while converting.

You can specify one of these units: "mm/s", "cm/s", "m/s", "km/h", "m/min", "in/s", "ft/s", "yd/s", or "mph".

Default Value of the Parameter

The default value is "m/s".

Return Value

The return value has the data type *speed*.

Example

```
var a: speed  
a := num_to_speed(4.1, "m/s")
```

See also

[speed_to_num \[SimTalk\]](#)

num_to_time [SimTalk]

Converts the specified number without a unit into a value of data type *time*.

Type

Function

Syntax

```
num_to_time(Number:real) → time
```

Parameter

The parameter *Number* of data type *real* designates the number which is going to be converted.

Return Value

The return value has the data type *time*.

Example

```
var s: speed  
s := Track.length / num_to_time(30)
```

num_to_weight [SimTalk]

Converts the specified number without a unit into a value of data type *weight*.

Type

Function

Syntax

```
num_to_weight(Number:real[, Unit:string="kg"]) → weight
```

Parameters

You can specify the following parameters:

- The parameter *Number* of data type *real* designates the number which is going to be converted.
- The optional parameter *Unit* of data type *string* designates the number without a value which is going to be converted to a physical data type by using the specified unit.

You can specify one of these units: "g", "kg", "t", "lb", or "oz".

Default Value of the Parameter

The default value is "kg".

Return Value

The return value has the data type *weight*.

Example

```
var w: weight  
w := num_to_weight(Track4.Length)
```

See also

[weight_to_num \[SimTalk\]](#)

Converting References

Converting References

SimTalk provides the functions listed in the table of contents to the left for converting references.

obj_to_str [SimTalk]

Converts the path and the name of the designated object to a value of data type *string*.

Type

Function

Syntax

```
obj_to_str(obj:object[, MakeAbsolute:boolean:=true]) -> string
```

Parameters

You can specify the following parameters:

- The parameter *obj* of data type *object* designates the path of the object to be returned as string.
- The optional parameter *MakeAbsolute* of data type *boolean*, sets if the absolute path is to be always returned (*true*). (This is the behavior of previous versions and thus is the default if you do not specify this parameter).

If you specify *false*, the function returns the actual path set in the first parameter. This can be a relative path.

Default Value

The default value is *true*.

Return Value

The return value has the data type *string*.

Example

```
var o1, o2 : object
var s : string
o1 := "Station.Cont.~"
s := obj_to_str(o1)           // ".Models.Model.Station"
s := obj_to_str(o1, false)   // "Station.Cont.~"
o2 := "Station.OnEntrance"   // path to a user-defined attribute of data
type method
s := obj_to_str(o2)           // "VOID"
s := obj_to_str(o2, false)   // "Station.OnEntrance"
```

str_to_method [SimTalk]

Checks if the path designated by the parameter *path* references an object of type *Method* or a *user-defined attribute* of data type *method*.

Remarks

If the path is a **relative path**, for example "`self.OnEntrance`", you can specify in the parameter at which object the path evaluation is to start.

Use the function *str_to_method* to access a control that is entered into an object.

Type

Function

Syntax

```
str_to_method(Path:any[, Context:object]) → object/method
```

Parameters

You can specify the following parameters:

- The parameter *Path* of data type *any* designates the path that is going to be evaluated.
The first parameter can be of data type *string* or of data type *object*. This is useful to evaluate a path from an attribute of an object, for example the exit control *ExitCtrl*. In this case you do not have to convert the value of the attribute to a *string*, using *to_str* before calling the function *str_to_method*.
- If you specify the optional parameter *Context* of data type *object* and if you specified a **relative path** as the parameter *Path*, the path evaluation of the relative path does not start in the surrounding *Frame* of the method, which contains the *str_to_method* statement, but in the specified object. When the relative path starts with `self`, `self` will be replaced with the second parameter. If the relative path does not start with `self`, the path evaluation starts with the surrounding *Frame* of the second parameter, unless the specified object itself is a *Frame*. Then the path evaluation starts there.

Return Value

The return value has the data type *object* or *method*.

It is the reference to the respective *Method*, otherwise it is VOID.

Examples

```
str_to_method(Station.ExitCtrl, Station).Program  
// returns the source code of the exit control
```

```
str_to_method(Station.ExitCtrl, Station).execute  
// executes the entered exit control
```

See also

[to_str \[SimTalk\]](#)

str_to_obj [SimTalk]

Converts the text, which designated an absolute or a relative path to an object, into an *object*.

Remarks

The string must designate an object, not a user-defined attribute. If you want to resolve a path to a user-defined table attribute or to a user-defined method attribute, you need to use *str_to_table* or *str_to_method* respectively.

Type

Function

Syntax

```
str_to_obj(Text:string) → object
```

Parameter

The parameter *Text* of data type *string* designates the text that you want to convert.

Return Value

The return value has the data type *object*.

void if the passed string does not designate an object.

Example

```
var o: object := str_to_obj(".Method")
```

See also

[str_to_table \[SimTalk\]](#)

[str_to_method \[SimTalk\]](#)

str_to_table [SimTalk]

Checks if the path designated by the parameter *path* references a *user-defined attribute* of data type *table*, *list*, *stack*, or *queue*.

Remarks

str_to_table a reference to this user-defined attribute, otherwise it returns VOID. If the path is a **relative path**, you can specify in the parameter *context* at which object the path evaluation is to start.

Syntax

```
str_to_table(Path:string/object[, Context:object]) → object/table/list/stack/queue
```

Parameters

You can specify the following parameters:

- The parameter *Path* of data type *string* or *object* designates the path that you want to evaluated.

Note

The first parameter can be of data type *string* or of data type *object*.

- If you specify the optional parameter *Context* of data type *object* and if you specified a **relative path** as the parameter *Path*, the path evaluation of the relative path does not start in the surrounding *Frame* of the method, which contains the *str_to_table* statement, but in the specified object. When the relative path starts with `self`, `self` will be replaced with the second parameter. If the relative path does not start with `self`, the path evaluation starts with the surrounding *Frame* of the second parameter, unless the specified object itself is a *Frame*. Then the path evaluation starts there.

Return Value

The return value has the data type *object* or *table*, *list*, *stack*, or *queue*.

- The return value has the data type *object* if the path points to a table or a list in the *Frame*.
- The return value has the data type *table*, *list*, *stack*, or *queue* if the path points to a subtable or a sublist or to a *user-defined attribute* of data type *table* or *list*.
- The return value is VOID if the path does not point to an object or a *user-defined attribute* or if the object or *user-defined attribute* is not of data type *table* or *list*.

Example

```
// returns the user-defined attribute of data type table of the Station
var anyVar:any := str_to_table(ListTypeTable, Station)
```

```
print anyVar  
print getSimTalkTypename(anyVar)
```

Converting Time and Date Values

Converting Time and Date Values

SimTalk provides the functions listed in the table of contents to the left for converting date and time values.

datetime_to_str [SimTalk]

Converts a specified value according to the format string in the string to a value of data type *string*.

Remarks

If you do not specify a format string, the result is the same as for the function *to_str*.

Type

Function

Syntax

```
datetime_to_str(Value:datetime[, Format:string])
```

Parameters

You can specify the following parameters:

- The parameter **Value** of data type *DateTime* designated the value that contains the date and time that you want to convert.
- The parameter **Format** of data type string designated the format string. The format string recognizes these special meanings:

%d: Day of month as decimal number (01 – 31)

%m: Month as decimal number (01 – 12)

%y: Year without century, as decimal number (00 – 99)

%Y: Year with century, as decimal number

%H: Hour in 24-hour format (00 – 23)

%h: Hour in 12-hour format (01 – 12)

%M: Minute as decimal number (00 – 59)

%S: Second as decimal number (00 – 59)

%s: Second as real number (00.0000 – 59.9999)

%p: a.m./p.m. indicator for 12-hour format

%P: AM/PM indicator for 12-hour format

%x: Date representation for current locale

%X: Time representation for current locale

Example

```
print datetime_to_str(str_to_datetime("2026/03/20 0:43:22"), "%d.%m.%y
%H:%M:%S")
-- 20.03.26 00:43:22

print datetime_to_str(str_to_datetime("2026/03/20 0:43:22"), "Date:
%d.%m.%y Time: %H:%M:%S")
-- Date: 20.03.26 Time: 00:43:22

print datetime_to_str(str_to_datetime("2026/01/31 13:10:23"), "%Y/%m/%d
%h:%M:%S %p")
-- 2026/01/31 01:10:23 p.m.
```

See also

[to_str \[SimTalk\]](#)

isValidDateString [SimTalk]

Returns if the passed value contains a valid date or not.

Type

Function

Syntax

```
isValidDateString(Date:string) → boolean
```

Parameter

The parameter *Date* of data type *string* designates the value of which you want to know if it contains a valid date. You can specify the year with two or with four digits. The function also takes leap years into account.

You can specify the date in these formats: "YYYY/MM/DD", "YYYY-MM-DD", and "DD.MM.YYYY".

Return Value

The return value has the data type *boolean*.

Example

```
print isValidDateString("2024/02/10")  
print isValidDateString("2024-02-10")  
print isValidDateString("10.02.2024")
```

See also

[to_str \[SimTalk\]](#)

isValidDateTimeString [SimTalk]

Returns if the passed value contains a valid date and time or not.

Type

Function

Syntax

```
isValidDateTimeString(DateAndTime:string) → boolean
```

Parameter

The parameter *DateAndTime* of data type *string* designates the value of which you want to know if it contains a valid date and time. You can specify the year with two or with four digits. The function takes leap years into account.

You can specify a date containing a time in these formats: "YYYY/MM/DD HH:MM:SS.s", "YYYY-MM-DD HH:MM:SS.s", and "DD.MM.YYYY HH:MM:SS.s". You can replace the space between date and time with the letter T.

Return Value

The return value has the data type *boolean*.

Example

```
print isValidDateTimeString("2024/02/10 14:00:00.00")
print isValidDateTimeString("2024-02-10 14:00:00.00")
print isValidDateTimeString("10.02.2024 14:00:00.00")
print isValidDateTimeString("10.02.2024T14:00:00.00")
```

See also

[str_to_dateTime \[SimTalk\]](#)

isValidTimeString [SimTalk]

Returns if the passed value contains a valid time (true) or not (false).

Syntax

```
isValidTimeString(Date:string) → boolean
```

Parameter

The parameter *Date* of data type *string* designates the value of which you want to know if it contains a valid time.

Return Value

The return value has the data type *boolean*.

Example

```
print isValidDateTimeString("14:00:00.00")
```

See also

[str_to_time \[SimTalk\]](#)

str_to_date [SimTalk]

Converts the specified date-statement into a statement of data type *date*.

Remarks

Conversion depends on the setting you selected under **File > Model Settings/Preferences > Units > Time Scale**.

Type

Function

Syntax

```
str_to_date(DateStatement:string) → date
```

Parameter

The parameter *DateStatement* of data type *string* designates the date that you want to convert. The year either has two or four digits. The function also takes leap years into account.

You can specify the date in these formats: "YYYY/MM/DD", "YYYY-MM-DD", and "DD.MM.YYYY".

Return Value

The return value has the data type *date*.

Example

```
print str_to_date("2024/02/10") // 2024/02/10
print str_to_date("2024-02-10") // 2024/02/10
print str_to_date("10.02.2024") // 2024/02/10
```

See also

[isValidDateString \[SimTalk\]](#)

[Time Scale \[preferences\]](#)

str_to_dateTime [SimTalk]

Converts the specified date-statement into a statement of data type *dateTime*.

Remarks

Conversion depends on the setting you selected under **File > Model Settings/Preferences > Units > Time Scale**.

Type

Function

Syntax

```
str_to_dateTime(DateTimeStatement:string) → dateTime
```

Parameter

The parameter *DateTimeStatement* of data type *string* designates the date and the time that you want to convert. The year either has two or four digits. The function also takes leap years into account.

You can specify a date containing a time in these formats: "YYYY/MM/DD HH:MM:SS.s", "YYYY-MM-DD HH:MM:SS.s", and "DD.MM.YYYY HH:MM:SS.s". You can replace the space between date and time with the letter T.

Return Value

The return value has the data type *dateTime*.

Example

```
print str_to_datetime("2024/02/10 14:00:00.00")
print str_to_datetime("2024-02-10 14:00:00.00")
print str_to_datetime("10.02.2024 14:00:00.00")
print str_to_datetime("10.02.2024T14:00:00.00")
```

See also

[isValidDateTimeString \[SimTalk\]](#)

[Time Scale \[preferences\]](#)

str_to_time [SimTalk]

Converts the specified data into a time-statement of data type *time*.

Remarks

Plant Simulation analyzes the statement from right to left. It has this format hh:mm:ss.ss. The value "1:00.1" stands for one minute plus one tenth of a second.

Note

Conversion depends on the setting you selected under **File > Model Settings/Preferences > Units > Time Scale**.

Type**Function****Syntax**

```
str_to_time(Data:string[, FormatLikeDialogs:boolean]) → time
```

Parameters

You can specify the following parameters:

- The parameter *Data* of data type *string* designates the data to be converted.
- The optional parameter *FormatLikeDialogs* of data type *boolean* sets if *Plant Simulation* uses the same time format as in text boxes of the dialogs (*true*). When you do not enter a value for the parameter or when you set it to *false*, *Plant Simulation* uses the same time format as in *tables*, which shows four decimal places.

Return Value

The return value has the data type *time*.

The statement `str_to_time("1:30.50")` returns different values, depending on the setting you selected under **File > Model Settings/Preferences > Units > Time Scale**.

Setting	Seconds
Time scale 1/1.0 Transfer If 24:60:60	90.5
Time scale 1/60.0 Transfer If 30:24:60	5430
Time scale 1/0.6 Transfer If 24:60:100	78.3

Example

```
var t: time
t := str_to_time("48:00:00.0") // 48 hours
self.executeIn(str_to_time("5:00")) // 5 minutes
```

See also

[isValidTimeString \[SimTalk\]](#)

[Time Scale \[preferences\]](#)

timeRepresentation [SimTalk]

Returns the preferences you selected for the time scale under **File > Model Settings/Preferences > Units > Time Scale**.

Type

Function

Syntax

```
timeRepresentation -> list
```

Return Value

The return value is a *DataList* of data type *real*.

The first entry contains the conversion factor for the time. You can only specify the denominator. The conversion factor is the reciprocal value. The following three entries are the values taken from **File > Model Settings/Preferences > Units > Time Scale**. They are normally represented by 60, 60 and 24.

Example

```
var l: list[real]
var timeDivision: real
var SecondCarriedOver, HourCarriedOver, DayCarriedOver : real
l := timeRepresentation
timeDivision := l.read(1)
secondCarriedOver := l.read(2)
hourCarriedOver := l.read(3)
dayCarriedOver := l.read(4)
```

See also

[Time Scale \[preferences\]](#)

Converting Arrays

Converting Arrays

SimTalk provides the functions listed in the table of contents to the left for converting arrays.

bytes_to_str [SimTalk]

Converts an *array* of *integer numbers* to text.

Remarks

Plant Simulation uses the specified encoding to interpret the sequence of numbers correctly and to create text from them.

Use *bytes_to_str* to convert application-specific or binary data received from the *MQTT Interface* or the http request functions to a string using a known code page or a code page detection.

You can use *bytes_to_str* in combination with *readBytesFromFile* instead of the function *readStringFromFile*. If the file to be read has an encoding, which is not supported by *readStringFromFile*, then *bytes_to_str* can work on the file content read by *readBytesFromFile* using the known encoding.

Type

Function

Syntax

```
bytes_to_str(Encoding:string/integer, Bytes:integer[]) -> string
```

Parameters

You can specify the following parameters:

- The parameter *Encoding* of data type *integer* sets the encoding to be used. This is an *integer* number specifying a Windows code page identifier, such as 28591 for ISO 8859-1 (Latin 1) or 1201 for UTF-16 Big Endian. Specifying an invalid integer number as code page identifier causes an error.
 - **ANSI** uses the encoding of the user interface language of *Plant Simulation* to interpret the *integers* as letters.
 - **UTF-8** uses UTF-8 encoding to interpret the *integers* as text.
 - **UTF-16** or **Unicode** encoding interprets the sequence of *integers* as Unicode text; two *integers*/bytes are always mapped to a single letter.
 - **BOM** checks the sequence of *integers* if it contains a Byte Order Mark (BOM) at the beginning and then uses the encoding derived from that to convert the rest of the *integers* to text. BOM supports the UTF-8 and UTF-16 (Unicode Little Endian) encodings, as they are supported across the application.
 - **Codepoints** uses a sequence of *integers* as a sequence of Unicode values; each number represents a Unicode letter. Numbers outside of the UTF-16 Unicode range cause an invalid argument error as do numbers that do not represent a valid letter in Unicode.

- The parameter *Bytes* is an *array* of data type *integer* containing the values which are going to be interpreted as a sequence of bytes or of Unicode code points.

Return Value

The return value has the data type *string*.

Example

```
var fileName:string := "c:\temp\helloWorld_UTF7.txt"
var bytes:integer[] := readStringFromFile(fileName)
print bytes
// prints [72, 101, 108, 108, 111, 32, 87, 111, 114, 108, 100]

print bytes_to_str(65000 /* UTF-7 */, bytes)
// prints Hello World
```

See also

[readBytesFromFile \[SimTalk\]](#)

[readStringFromFile \[SimTalk\]](#)

Functions for Communicating with an HTTP Server